

Object Detection & Image Segmentation

Structured Notes

YOLO Family (v1–v6), SSD,

Instance/Binary/Multi-class Segmentation,
U-Net and U-Net++ Architectures

Contents

1	Overview of Object Detection Approaches	2
2	YOLO Family of Object Detectors	2
2.1	YOLO Version 1 (2016)	2
2.1.1	Core Mechanism	2
2.1.2	Performance Characteristics	3
2.1.3	Limitations	3
2.2	YOLO Version 2 / YOLO 9000 (2016)	3
2.2.1	Improvement 1: Larger Input Resolution	3
2.2.2	Improvement 2: Batch Normalization	3
2.2.3	Improvement 3: Anchor Boxes	3
2.3	YOLO Version 3 (2018)	4
2.3.1	Darknet-53 Backbone	4
2.3.2	Multi-Scale Detection (Feature Pyramid)	4
2.3.3	Smoother Bounding Box Prediction	4
2.4	YOLO Version 4 (2020)	4
2.4.1	Architecture Overview	4
2.4.2	CSP Darknet (Cross Stage Partial Network)	4
2.4.3	Bag of Freebies	5
2.4.4	PANet (Path Aggregation Network)	5
2.5	YOLO Version 5 (2020)	5
2.5.1	Model Variants	5
2.5.2	Key Features	5
2.6	YOLO Version 6 (2022)	5
2.6.1	Performance	6
2.7	YOLO Version Selection Guidelines	6
3	IOU Score and Non-Maximum Suppression (NMS)	6
3.1	IOU (Intersection Over Union)	6
3.2	Non-Maximum Suppression (NMS)	6
3.2.1	NMS Algorithm	6
4	SSD: Single Shot Multi-Box Detector	7
4.1	Multi-Scale Feature Maps	7
4.2	Default Anchor Boxes	7
4.3	Advantages Over YOLO	7
5	Image Segmentation	7
5.1	Types of Segmentation	8
5.2	Challenges in Image Segmentation	8
5.3	Medical Imaging Application: Brain Tumor Segmentation	8
5.4	Why Deep Learning for Segmentation	9
6	U-Net Architecture	9
6.1	Skip Connections	9
6.2	Example Spatial Dimensions	10
7	U-Net++ Architecture	10
7.1	The Semantic Gap Problem in U-Net	10
7.2	Dense Skip Connections	10
7.3	Advantages of U-Net++	10
8	Summary	11

1 Overview of Object Detection Approaches

Object detection involves identifying and localizing objects within images or video frames. There are two fundamental categories of bounding box detectors: **two-stage detectors** and **one-stage detectors**.

Two-Stage Object Detection

In two-stage object detection, the process is divided into two sequential phases:

1. **Region Proposal Generation:** The input image is fed to a feature extraction module that generates proposals — regions where there is a high probability of an object being present.
2. **Classification and Regression:** The proposed regions are passed through a classification block (to determine the category of the object) and a regression block (to determine the precise location/position of the object in the frame).

Examples include R-CNN, Fast R-CNN, and Faster R-CNN.

One-Stage Object Detection

In one-stage object detection, the input image is fed to a feature extraction module and classification plus detection of objects happens **directly** — without any intermediate proposal generation step. The model simultaneously predicts bounding box locations and class labels in a single forward pass.

Examples include YOLO (You Only Look Once) and SSD (Single Shot Multi-Box Detector).

Key Distinction

One-stage detectors skip the proposal generation step entirely, going directly from feature extraction to object classification and localization. This makes them significantly faster than two-stage detectors, though traditionally with some trade-off in accuracy.

2 YOLO Family of Object Detectors

YOLO (You Only Look Once) is a family of one-stage object detection algorithms. Each successive version introduces architectural improvements to address limitations of prior versions.

2.1 YOLO Version 1 (2016)

YOLO v1 was developed in 2016 and introduced the grid-based detection paradigm.

2.1.1 Core Mechanism

The input image of size $M \times N$ is divided into a grid of $S \times S$ cells. Each grid cell predicts:

- **Bounding boxes** for objects whose center falls within that cell
- **Probability scores** for each object class

The model jointly encodes both the object location (via regression) and the category information (via classification).

YOLO v1 Pipeline:

Input Image $\xrightarrow{\text{Resize}}$ $S \times S$ Grid $\xrightarrow{\text{CNN}}$ Bounding Box + Class Probability

Each grid cell predicts B bounding boxes and C class probabilities simultaneously.

2.1.2 Performance Characteristics

- **Speed:** 150 frames per second (FPS) — very high inference speed
- **Accuracy:** 63% on Pascal VOC 2012 dataset — below state-of-the-art models like Mask R-CNN

2.1.3 Limitations

- Struggles with **small objects** due to coarse grid division
- Difficulty with **overlapping objects** in the same grid cell
- Lower accuracy compared to two-stage detectors like Mask R-CNN

2.2 YOLO Version 2 / YOLO 9000 (2016)

YOLO v2 (also called YOLO 9000) was proposed by J. Redmon and A. Farhadi in 2016 as an improved version of YOLO v1, achieving **78.6% accuracy** on Pascal VOC 2012 (up from 63%).

2.2.1 Improvement 1: Larger Input Resolution

Aspect	YOLO v1	YOLO v2
Input Size	224×224	416×416 (or 448×448)
Output Feature Map	Limited resolution	$13 \times 13 \times 3072$
Small Object Detection	Poor	Improved ($\sim 4\%$ gain)

By using a larger input image (416×416 instead of 224×224), even after downsampling through convolution and pooling operations to 13×13 , more spatial information is preserved. This enables detection of smaller objects that were missed by YOLO v1.

2.2.2 Improvement 2: Batch Normalization

YOLO v2 introduced **batch normalization** after every convolutional layer to address the vanishing gradient problem that arises with deep networks.

Batch Normalization:

$$\hat{x} = \frac{x - \mu}{\sigma}$$

where μ is the mean and σ is the standard deviation computed over the batch at each layer. This normalizes the output of every layer, leading to:

- Faster convergence during training
- Mitigation of vanishing gradient problem
- Approximately 2% accuracy improvement

2.2.3 Improvement 3: Anchor Boxes

Anchor Boxes vs Bounding Boxes

- **Bounding boxes** (YOLO v1): Random-sized boxes placed during detection; the model predicts box dimensions from scratch.
- **Anchor boxes** (YOLO v2): **Predefined templates** with specific sizes and aspect ratios matched to common object shapes (e.g., tall rectangle for humans, wide rectangle for cars).

Using anchor boxes provides the detector with prior knowledge about typical object shapes, making it more robust to varying object sizes and improving IOU scores.

This change contributed approximately 10% accuracy improvement, bringing the total improvement of YOLO v2 over v1 to **15.6% on Pascal VOC 2012**.

2.3 YOLO Version 3 (2018)

YOLO v3 was proposed by J. Redmon and A. Farhadi in 2018, addressing speed and accuracy limitations of v1 and v2.

2.3.1 Darknet-53 Backbone

YOLO v3 uses **Darknet-53** as its backbone — a CNN architecture consisting of 53 convolutional layers.

- Uses only 3×3 and 1×1 convolution filters (small kernels preserve fine-grained details and small objects)
- Includes **residual connections** between layers — feature maps from earlier layers are added back to later layers
- Residual connections mitigate vanishing gradient in the 53-layer deep network (batch normalization alone is insufficient at this depth)
- The deep architecture enables stronger feature extraction at multiple levels

2.3.2 Multi-Scale Detection (Feature Pyramid)

Rather than extracting features at a single scale, YOLO v3 extracts features at **multiple scales**:

1. Original image size (e.g., 448×448)
2. Downsampled to 224×224
3. Further downsampled to 112×112 , 56×56 , etc.

Additionally, features from lower resolution levels are **upsampled** and combined with higher resolution feature maps. Convolution is applied at different scales, improving performance on both small and medium-sized objects.

2.3.3 Smoother Bounding Box Prediction

YOLO v3 uses an **independent logistic classifier** with multi-level prediction for bounding box regression. This smoother regression approach improves the accuracy of predicting object locations compared to previous versions.

YOLO v3 Summary

Darknet-53 backbone (53 layers with residual connections) + multi-scale feature pyramid + improved bounding box regression = significantly better accuracy and maintained real-time performance.

2.4 YOLO Version 4 (2020)

YOLO v4 was proposed by Alexey Bochkovskiy in 2020, targeting efficient real-time object detection on resource-constrained hardware (e.g., edge devices in border surveillance where large GPU infrastructure is unavailable).

2.4.1 Architecture Overview

YOLO v4 follows a structured **Input** → **Backbone** → **Neck** → **Head** pipeline:

Component	Description
Input	The input image
Backbone	CSP Darknet — feature extraction network
Neck	PANet — fuses multi-scale features for better localization
Head (Dense)	Dense prediction layer for object location
Head (Sparse)	Sparse prediction for precise object localization

2.4.2 CSP Darknet (Cross Stage Partial Network)

Cross Stage Partial Network (CSPNet)

CSPNet reduces computational redundancy by **splitting features into multiple groups** for parallel processing rather than processing all features through every layer sequentially. Key advantages:

- **Improved gradient flow** through partial residual connections
- **Backbone flexibility** — can use VGG, ResNet, ResNeXt, DenseNet, or other architectures as the backbone (not limited to Darknet-53)
- **Reduced computational cost** while maintaining feature quality

2.4.3 Bag of Freebies

YOLO v4 is also referred to as a “**Bag of Freebies**” model because the CSPNet backbone provides flexibility to use different types of features for the training process without relying solely on Darknet-53. Various data augmentation and training strategies can be applied without increasing inference cost.

2.4.4 PANet (Path Aggregation Network)

Since the flexible backbone can produce features of different dimensions from different networks, **PANet** is used to fuse features across multiple scales:

- Combines low-level features (small objects, spatial details) with high-level features (large objects, semantic information)
- Strengthens feature representation for improved localization
- Reduces information loss in deep feature networks

2.5 YOLO Version 5 (2020)

YOLO v5 is a **PyTorch-based** reimplementation and improvement of the YOLO v3 architecture, offering engineering efficiency and multiple model sizes.

2.5.1 Model Variants

YOLO v5 comes in four sizes to suit different deployment requirements:

Variant	Use Case
Small (S)	Edge devices, mobile
Medium (M)	Balanced speed/accuracy
Large (L)	High accuracy applications
Extra Large (XL)	Maximum accuracy

2.5.2 Key Features

- **Architecture:** Uses CSP Darknet-53 backbone with Spatial Pyramid Pooling (SPP) for faster performance
- **Data Augmentation:** Employs **Mosaic** and **Mix-up** augmentation techniques — combining multiple images through tearing, shearing, shifting, and other transformations to increase training data diversity
- **Custom Anchor Boxes:** Improved anchor box design for better object detection
- **PyTorch Native:** Full PyTorch implementation enabling easy customization and deployment

2.6 YOLO Version 6 (2022)

YOLO v6 was developed in 2022, specifically designed for **industrial and edge-based deployment**.

Edge Analytics vs Web-Based Analytics

- **Web-Based Analytics:** Camera footage is sent to a centralized server where the YOLO algorithm processes the recorded data. This introduces latency due to data transfer and storage.
- **Edge Analytics:** Cameras are equipped with small processors/sensors that run object detection algorithms **directly at the camera level**, eliminating the need to send data to a central server. Processing happens in real-time at the “edge” of the network.

2.6.1 Performance

- Achieves up to **1242 FPS** inference speed for smaller networks
- Handles up to **520 FPS** for larger datasets
- Designed for 24/7/365 continuous operation in industrial settings
- Uses **PANet** architecture for feature fusion

2.7 YOLO Version Selection Guidelines

Choosing the Right YOLO Version

The choice of YOLO version depends on the specific application requirements:

- **Hardware constraints:** If limited GPU resources are available, use a lighter version (v5 Small, v6)
- **Data characteristics:** Consider the size and type of objects in the data
- **Deployment environment:** Edge vs cloud deployment
- Where YOLO v3 is sufficient for the architecture, using YOLO v8 may be unnecessarily costly
- The latest available version in the market is YOLO v8, though many practitioners find v3, v4, or v5 adequate for their applications

3 IOU Score and Non-Maximum Suppression (NMS)

3.1 IOU (Intersection Over Union)

IOU is the metric used to evaluate how well a predicted bounding box matches the ground truth.

$$\text{IOU} = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

An IOU score closer to 1.0 indicates a better match between the predicted and actual bounding box.

3.2 Non-Maximum Suppression (NMS)

NMS is applied to eliminate redundant overlapping bounding boxes and retain only the most accurate detection for each object.

3.2.1 NMS Algorithm

1. For all detected bounding boxes, compute IOU scores
2. Sort all boxes in **decreasing order** of IOU (e.g., 0.98, 0.93, 0.92, 0.9, ...)
3. Select the box with the **highest IOU score** (e.g., 0.98)
4. Compute IOU of this selected box with all remaining boxes
5. **Suppress (remove)** any box that has a high IOU overlap with the selected box — these represent the same object
6. Boxes representing **different objects** (low IOU overlap with the selected box) are retained
7. Repeat the process for the remaining boxes

NMS and Segmentation

NMS is used specifically for **object detection** (bounding box localization). It is **not used** for object segmentation, which employs different architectures such as encoder-decoder models.

4 SSD: Single Shot Multi-Box Detector

SSD is another one-stage detection approach that predicts bounding boxes from **multiple feature maps** at different scales.

SSD Architecture

SSD stands for **Single Shot Multi-Box Detector**. Unlike YOLO which uses a single feature map for detection, SSD:

1. Uses a base feature extraction network (VGG, ResNet, or other architectures)
2. Appends **extra convolutional layers** to generate additional feature maps at different spatial resolutions
3. Applies **default anchor boxes at multiple levels** of the feature hierarchy

4.1 Multi-Scale Feature Maps

The key innovation of SSD is applying detection at **multiple scales simultaneously**:

- Rather than using a single anchor box on a particular feature map, SSD places **multiple anchor boxes at different feature extraction levels**
- Additional convolutional layers are appended to the base network to generate feature maps with different spatial resolutions
- Each feature map level handles objects of different sizes — larger feature maps detect smaller objects, smaller feature maps detect larger objects

4.2 Default Anchor Boxes

At each location in each feature map, SSD places a set of **default anchor boxes** with predefined aspect ratios. The network then predicts:

- Offsets to adjust each default box to better fit the actual object
- Confidence scores for each object category

4.3 Advantages Over YOLO

- **Multi-resolution detection:** Features extracted at different spatial resolutions provide more accurate predictions for objects of varying sizes
- **Real-time performance:** Maintains single-stage detection speed
- **Flexible backbone:** Can use VGG, ResNet, or other architectures as the base feature extractor
- NMS is applied on top of the multi-scale detections to produce final results

5 Image Segmentation

After covering bounding box-based object detection, the next major topic is **image segmentation** — the task of assigning labels to individual pixels rather than drawing bounding boxes around objects.

5.1 Types of Segmentation

Type	Description
Instance Segmentation	Separates individual objects of the same class from each other. For example, if multiple persons are in an image, each person is segmented as a distinct instance with pixel-level boundaries.
Binary Segmentation	Divides the image into foreground and background . One class represents the object of interest, everything else is background. Example: tumor vs. non-tumor in brain MRI.
Multi-Class Segmentation	Assigns each pixel to one of multiple predefined categories — road, footpath, sky, mountain, car, human, tree, etc.
Semantic Segmentation	Assigns each pixel a class label from predefined categories (e.g., bird, cat, dog). All pixels of the same class get the same label, without distinguishing between individual instances.

Instance vs Semantic Segmentation

- **Semantic segmentation:** All pixels belonging to “person” get the same label, regardless of which person they belong to
- **Instance segmentation:** Each individual person is treated as a separate entity with a unique label — Person 1 (wrestler), Person 2 (worker), Person 3, etc.

Segmentation can be both **supervised** (labels specify object classes: person, bird, cat) and **unsupervised** (clusters are formed without predefined class names).

5.2 Challenges in Image Segmentation

1. **Precise boundary localization:** The segmentation algorithm must accurately identify the exact boundary of each object
2. **Scale variation:** Objects may range from very small to very large; models trained on one scale may fail on another
3. **Illumination changes:** Models trained under good lighting conditions may fail when illumination changes significantly
4. **Complex backgrounds:** A model trained to detect a crocodile on transparent water may fail when the background is muddy — background complexity degrades performance
5. **Occlusion and shadows:** Partially hidden objects and shadows create additional segmentation challenges

5.3 Medical Imaging Application: Brain Tumor Segmentation

Precise segmentation is critically important in medical imaging, particularly for brain tumor detection via MRI.

Under-Segmentation and Over-Segmentation

- **Under-segmentation:** The algorithm detects a smaller region than the actual tumor. During laser-guided surgery, the undetected portion of the tumor remains, leading to potential regrowth.
- **Over-segmentation:** The algorithm marks a larger region than the actual tumor. Healthy brain tissue is unnecessarily removed during surgery, causing damage to non-affected areas.

Both scenarios have severe clinical consequences, motivating the need for highly accurate deep

learning-based segmentation models.

5.4 Why Deep Learning for Segmentation

Traditional approaches using manual feature extraction (edge, texture, intensity) are highly affected by:

- Light changes, noise, and occlusion
- Varying illumination and scale
- Object orientation changes
- Multiple processing pipeline stages (preprocessing → feature extraction → segmentation → classification)

Deep learning models provide a **single end-to-end framework** where:

- Class labels are provided as input to the model
- The same model architecture can handle different segmentation tasks (brain MRI, autonomous driving, etc.)
- The model works on heterogeneous data without task-specific feature engineering

6 U-Net Architecture

U-Net is the primary deep learning architecture used for image segmentation. It has a symmetric **U-shaped structure** consisting of an encoder (contracting path) and a decoder (expanding path).

U-Net Architecture

U-Net consists of three main components:

1. **Encoder (Contracting Path):** The left side of the U-shape. The input image is passed through multiple layers of convolution and downsampling (pooling) to extract increasingly abstract features. Each downsampling step reduces spatial dimensions while increasing the number of feature channels.
2. **Bottleneck:** The bottom of the U-shape. Contains the most compressed representation of the input, passing only the most important features from the encoder to the decoder.
3. **Decoder (Expanding Path):** The right side of the U-shape. Features from the bottleneck are progressively upsampled and convolved to reconstruct the output at the original image resolution ($M \times N$). The decoder is trained on class labels to segment specific regions.

6.1 Skip Connections

A critical feature of U-Net is the **skip connections** between corresponding encoder and decoder layers. These connections directly transfer feature maps from the encoder to the decoder at each resolution level, helping the decoder recover spatial information lost during downsampling.

U-Net Data Flow:

Input $\xrightarrow{\text{Conv} + \text{Pool}}$ Encoder Level 1 $\xrightarrow{\text{Conv} + \text{Pool}}$... $\xrightarrow{\text{Conv} + \text{Pool}}$ Bottleneck

Bottleneck $\xrightarrow{\text{UpConv} + \text{Concat}}$ Decoder Level 1 $\xrightarrow{\text{UpConv} + \text{Concat}}$... $\xrightarrow{\text{UpConv}}$ Output ($M \times N$)

At each decoder level, features from the corresponding encoder level are concatenated via skip connections.

6.2 Example Spatial Dimensions

Level	Encoder (Downsampling)	Decoder (Upsampling)
Level 1	256×256	256×256
Level 2	128×128	128×128
Level 3	64×64	64×64
Level 4	32×32	32×32
Bottleneck	16×16	—

7 U-Net++ Architecture

U-Net++ is an enhanced version of U-Net that addresses the **semantic gap** problem in skip connections.

7.1 The Semantic Gap Problem in U-Net

In standard U-Net, skip connections directly link encoder and decoder layers at the same spatial resolution. However, there is a significant **semantic gap** between these features:

- Encoder features at level 1 (256×256) are low-level (edges, textures)
- Decoder features at the same resolution have been processed through the bottleneck and carry high-level semantic information
- Direct concatenation of these semantically different features limits the network's ability to reconstruct accurate segmentations

7.2 Dense Skip Connections

U-Net++ solves this by introducing **intermediate dense skip connections** between encoder and decoder:

U-Net++ Dense Skip Connections

Instead of connecting encoder features directly to the corresponding decoder level, U-Net++ introduces intermediate nodes:

- Encoder features are progressively **upsampled through intermediate layers** before being connected to the decoder
- For example, rather than connecting 100×100 encoder features directly to the 100×100 decoder layer, the architecture:
 1. Downsamples $100 \times 100 \rightarrow 50 \times 50$
 2. Upsamples 50×50 back to 100×100 with learned transformations
 3. Similarly processes features from 25×25 , 12×12 , and 6×6 levels
 4. Concatenates all these intermediate representations
- This creates a **sequence of intermediate connections** that progressively bridge the semantic gap

7.3 Advantages of U-Net++

- **Reduced semantic gap:** Intermediate nodes transform features gradually, ensuring that concatenated features are semantically compatible
- **Dense feature reuse:** Features at every level are processed and combined, preventing information loss
- **Better segmentation accuracy:** The richer feature fusion leads to more precise boundary delineation
- **Flexible depth:** The nested architecture allows pruning for faster inference when full depth is not needed

8 Summary

-
- **Object detection** approaches are divided into **one-stage** (YOLO, SSD) and **two-stage** (R-CNN family) detectors
 - **YOLO v1** introduced grid-based detection at 150 FPS with 63% accuracy on Pascal VOC 2012
 - **YOLO v2** improved to 78.6% accuracy through larger input resolution (416×416), batch normalization, and anchor boxes
 - **YOLO v3** introduced Darknet-53 backbone with residual connections and multi-scale detection via feature pyramids
 - **YOLO v4** used CSP Darknet with PANet for flexible, efficient detection on edge devices
 - **YOLO v5** brought PyTorch-native implementation with four size variants (S/M/L/XL) and data augmentation (mosaic, mix-up)
 - **YOLO v6** targeted industrial edge deployment with up to 1242 FPS inference speed
 - **SSD** uses multi-scale feature maps with default anchor boxes at different levels for single-shot detection
 - **NMS** (Non-Maximum Suppression) eliminates redundant bounding boxes by suppressing overlapping detections with lower IOU scores
 - **Image segmentation** includes instance, binary, multi-class, and semantic segmentation types
 - **U-Net** uses an encoder-decoder architecture with skip connections for pixel-level segmentation
 - **U-Net++** improves upon U-Net by adding dense intermediate skip connections to bridge the semantic gap between encoder and decoder features